

Лабораторная работа по MySQL №1

Создание базы данных

1. Запустим сервер MySQL:

```
Z:\usr\local\mysql5\bin>mysql -u root -p
```

Система предлагает ввести пароль пользователя root MySQL, который был задан при установке MySQL (обычно он пустой по умолчанию). (Примечание: Это пароль пользователя root системы MySQL, а не пользователя root системы Linux). Введите пароль, который не изображается на экране по соображениям безопасности.

После успешной регистрации, система выводит приветствие и приглашение mysql.

```
Enter password:
```

```
Welcome to the MySQL monitor.  Commands end with ; or \g.
```

```
Your MySQL connection id is 1
```

```
Server version: 5.0.45-community-nt MySQL Community Edition (GPL)
```

```
Type 'help;' or '\h' for help. Type '\c' to clear the buffer.
```

```
mysql>
```

(Вас приветствует монитор MySQL. Команды заканчиваются символами ; или \g. id соединения с MySQL равен 1 для сервера версии: 5.01.01. Введите 'help', чтобы получить справку).

2. Теперь можно создавать базу данных **employees**. Выполните команду:

```
create database employees;
```

(Примечание: команда заканчивается точкой с запятой)

3. Ввод команды SHOW DATABASES; выведет список всех доступных в системе баз данных.

```
mysql> SHOW DATABASES;
```

Введите quit в строке приглашения mysql>, чтобы выйти из программы клиента mysql.

Команда CREATE DATABASE

Синтаксис команды CREATE DATABASE имеет вид:

```
CREATE DATABASE [IF NOT EXISTS] имя_базы_данных  
[спецификация_create[, спецификация_create] ...]
```

Команда CREATE DATABASE создает базу данных с указанным именем. Для использования команды необходимо иметь привилегию CREATE для базы данных. Если база данных с таким именем существует, генерируется ошибка.

спецификация_create:

```
[DEFAULT] CHARACTER SET имя_набора_символов  
[DEFAULT] COLLATE имя_порядка_сопоставления
```

Опция спецификация_create может указываться для определения характеристик базы данных. Характеристики базы данных сохраняются в файле db.opt, расположенном в каталоге данных. Конструкция CHARACTER SET определяет набор символов для базы данных по умолчанию. Конструкция COLLATION задает порядок сопоставления по умолчанию.

Базы данных в MySQL реализованы в виде каталогов, которые содержат файлы, соответствующие таблицам базы данных. Поскольку изначально в базе нет никаких таблиц, оператор CREATE DATABASE только создает подкаталог в каталоге данных MySQL.

Работа с таблицами

Теперь рассмотрим команды MySQL для создания таблиц базы данных и выбора базы данных.

Базы данных хранят данные в таблицах. Чем же являются эти таблицы?

Проще всего таблицы можно представлять себе, как состоящие из строк и столбцов. Каждый столбец определяет данные определенного типа. Строки содержат отдельные записи.

Рассмотрим таблицу, в которой приведены персональные данные некоторых людей:

Имя	Возраст	Страна	e-mail
Михаил Петров	28	Россия	misha@yandex.ru
Джон Доусон	32	Австралия	j.dow@australia.com
Морис Дрюон	48	Франция	md@france.fr
Снежана	19	Болгария	sneg@bulgaria.com

Приведенная выше таблица содержит четыре столбца, в которых хранятся имя, возраст, страна, и адрес e-mail. Каждая строка содержит данные одного человека. Эта строка называется записью. Чтобы найти страну и адрес e-mail Снежаны, сначала надо выбрать имя в первом столбце, а затем посмотреть содержимое третьего и четвертого столбцов этой же строки.

База данных может содержать множество таблиц, именно таблицы содержат реальные данные.

Следовательно, можно выделить связанные (или несвязанные) данные в различные таблицы. Для базы данных `employees` определена одна таблица, которая содержит данные компании о сотрудниках, а другая таблица будет содержать персональные данные. Давайте создадим первую таблицу.

Команда SQL для создания такой таблицы выглядит следующим образом:

```
CREATE TABLE employee_data
(
emp_id int unsigned not null auto_increment primary key,
f_name varchar(20),
l_name varchar(20),
title varchar(30),
age int,
yos int,
salary int,
perks int,
email varchar(60)
);
```

Примечание: в MySQL команды и имена столбцов не различают регистр символов, однако имена таблиц и баз данных могут зависеть от регистра в связи с используемой платформой (как в Linux). Поэтому можно вместо `CREATE TABLE` использовать `create table`.

За ключевыми словами `CREATE TABLE` следует имя создаваемой таблицы `employee_data`. Каждая строка внутри скобок представляет один столбец. Эти столбцы хранят для каждого сотрудника идентификационный номер (`emp_id`), фамилию (`l_name`), имя (`f_name`), должность (`title`), возраст (`age`), стаж работы в компании (`yos`), зарплату (`salary`), надбавки (`perks`), и адрес e-mail (`email`).

За именем каждого столбца следует тип столбца. Типы столбцов определяют тип данных, которые будет содержать столбец. В данном примере столбцы `f_name`, `l_name`, `title` и `email` будут содержать текстовые строки, поэтому тип столбца задан как `varchar`, что означает переменное количество символов. Максимальное число символов для столбцов `varchar` определяется числом, заключенным в скобки, которое следует сразу за именем столбца. Столбцы `age`, `yos`, `salary` и `perks` будут содержать числа (целые), поэтому тип столбца задается как `int`. Первый столбец (`emp_id`) содержит идентификационный номер (`id`) сотрудника. Его тип столбца выглядит несколько перегруженным, поэтому рассмотрим его по частям:

- **int**: определяет тип столбца как целое число.
- **unsigned**: определяет, что число будет без знака (положительное целое).
- **not null**: определяет, что значение не может быть `null` (пустым); то есть каждая строка в этом столбце должна иметь значение.

- **auto_increment**: когда MySQL встречается со столбцом с атрибутом **auto_increment**, то генерируется новое значение, которое на единицу больше чем наибольшее значение в столбце. Поэтому мы не должны задавать для этого столбца значения, MySQL генерирует их самостоятельно. Из этого также следует, что каждое значение в этом столбце будет уникальным.
- **primary key**: помогает при индексировании столбца, что ускоряет поиск значений. Каждое значение должно быть уникально. Ключевой столбец необходим для того, чтобы исключить возможность совпадения данных. Например, два сотрудника могут иметь одно и то же имя, и тогда встанет проблема — как различать этих сотрудников, если не задать им уникальные идентификационные номера. Если имеется столбец с уникальными значениями, то можно легко различить две записи. Лучше всего поручить присваивание уникальных значений самой системе MySQL.

Использование базы данных

База данных **employees** уже создана. Для работы с ней, необходимо её "активировать" или "выбрать". В приглашении `mysql` выполните команду:

```
SELECT DATABASE();
```

На экране увидите ответ системы. Это говорит о том, что ни одна база данных не была выбрана. На самом деле всякий раз при работе с клиентом `mysql` необходимо определять, какая база данных будет использоваться.

Определить текущую базу данных можно несколькими способами:

- определение имени базы данных при запуске

Введите в приглашении системы следующее:

```
mysql employees
```

- определение базы данных с помощью оператора **USE** в приглашении `mysql`

```
USE employees;
```

Определение базы данных с помощью `\u` в приглашении `mysql`

```
\u employees;
```

При работе необходимо определять базу данных, которая будет использоваться, иначе MySQL будет порождать ошибку.

Создание таблицы

После выбора базы данных **employees**, выполните в приглашении `mysql` команду **CREATE TABLE**.

```
CREATE TABLE employee_data
(
emp_id int unsigned not null auto_increment primary key,
f_name varchar(20),
l_name varchar(20),
title varchar(30),
age int,
yos int,
salary int,
perks int,
email varchar(60)
);
```

Примечание: нажатие клавиши **Enter** после ввода первой строки изменяет приглашение `mysql` на `->`. Это означает, что `mysql` понимает, что команда не завершена и приглашает ввести дополнительные операторы. Помните, что каждая команда `mysql` заканчивается точкой с запятой, а каждое объявление столбца отделяется запятой. Можно также при желании ввести всю команду на одной строке.

Синтаксис команды CREATE TABLE

Общий формат инструкции CREATE TABLE таков:
CREATE [TEMPORARY] TABLE [IF NOT EXISTS] имя
[(спецификация, ...)]
[опция, ...]
[[IGNORE | REPLACE] запрос]

Флаг TEMPORARY задает создание временной таблицы, существующей в течение текущего сеанса. По завершении сеанса таблица удаляется. Временным таблицам можно присваивать имена других таблиц, делая последние временно недоступными. Спецификатор IF NOT EXIST подавляет вывод сообщений об ошибках в случае, если таблица с указанным именем уже существует. Имени таблицы может предшествовать имя базы данных, отделенное точкой. Если это не сделано, таблица будет создана в базе данных, которая установлена по умолчанию.

Чтобы задать имя таблицы с пробелами, необходимо заключить его в обратные кавычки, например 'courses list'. То же самое нужно будет делать во всех ссылках на таблицу, поскольку пробелы используются для разделения идентификаторов.

Разрешается создавать таблицы без столбцов, однако в большинстве случаев спецификация хотя бы одного столбца все же присутствует. Спецификации столбцов и индексов приводятся в круглых скобках и разделяются запятыми. Формат спецификации следующий:

имя тип
[NOT NULL | NULL]
[DEFAULT значение]
[AUTO_INCREMENT]
[KEY]
[ссылка]

Спецификация типа включает название типа и его размерность. По умолчанию столбцы принимают значения NULL. Спецификатор NOT NULL запрещает подобное поведение.

У любого столбца есть значение по умолчанию. Если оно не указано, программа MySQL выберет его самостоятельно. Для столбцов, принимающих значения NULL, значением по умолчанию будет NULL, для строковых столбцов — пустая строка, для численных столбцов — нуль. Изменить эту установку позволяет предложение DEFAULT.

Поля-счетчики, создаваемые с помощью флага AUTO_INCREMENT, игнорируют значения по умолчанию, так как в них записываются порядковые номера. Тип счетчика должен быть беззнаковым целым. В таблице может присутствовать лишь одно поле-счетчик. Им не обязательно является первичный ключ.

Удаление таблиц

Для того, чтобы удалить таблицу, убедимся сперва, что она существует. Это можно проверить с помощью команды SHOW TABLES.

Для удаления таблицы используется команда DROP TABLE.

Теперь команда SHOW TABLES; этой таблицы больше не покажет.

Синтаксис команды DROP TABLE

Инструкция DROP TABLE имеет следующий синтаксис:

DROP TABLE [IF EXISTS] таблица [RESTRICT | CASCADE]

Спецификация IF EXISTS подавляет вывод сообщения об ошибке, выдаваемого в случае, если заданная таблица не существует. Можно указывать несколько имен таблиц, разделяя их запятыми.

Флаги RESTRICT и CASCADE предназначены для выполнения сценариев, созданных в других СУБД.

Запись данных в таблицы

Оператор INSERT заполняет таблицу данными. Вот общая форма INSERT.

```
INSERT into table_name (column1, column2, ...) values (value1, value2...);
```

где `table_name` является именем таблицы, в которую надо внести данные; `column1`, `column2` и т.д. являются именами столбцов, а `value1`, `value2` и т.д. являются значениями для соответствующих столбцов.

Следующий оператор вносит первую запись в таблицу `employee_data`.

```
INSERT INTO employee_data (f_name, l_name, title, age, yos, salary, perks, email)
values ("Михаил", "Петров", "директор", 28, 4, 200000, 50000, "misha@yandex.ru");
```

Как и другие операторы MySQL, эту команду можно вводить на одной строке или разместить ее на нескольких строках.

Несколько важных моментов: Значениями для столбцов `f_name`, `l_name`, `title` и `email` являются текстовые строки, и они записываются в кавычках. Значениями для `age`, `yos`, `salary` и `perks` являются числа (целые), и они не имеют кавычек. Можно видеть, что данные заданы для всех столбцов кроме `emp_id`. Значение для этого столбца задает система MySQL, которая находит в столбце наибольшее значение, увеличивает его на единицу, и вставляет новое значение.

Если приведенная выше команда правильно введена в приглашении клиента `mysql`, то программа выведет сообщение об успешном выполнении.

Создание дополнительных записей требует использования отдельных операторов INSERT. Чтобы облегчить эту работу можно поместить все операторы INSERT в файл. Это должен быть обычный текстовый файл с оператором INSERT в каждой строке. Заполнение таблицы `employee_data` данными с помощью файла `employee.dat`

1. Поместите файл в каталог `...\mysql\bin`.
2. Проверьте, что MySQL работает.
3. Выполните команду

```
mysql.exe -uroot -ppassword employees < employee.dat
```

Пусть таблица содержит теперь 21 запись (20 из файла `employee.dat` и одну, вставленную оператором INSERT в начале).

Запрос данных из таблицы MySQL

Таблица `employee_data` содержит теперь достаточно данных, чтобы можно было начать с ней работать. Запрос данных выполняется с помощью команды MySQL SELECT. Оператор SELECT имеет следующий формат:

```
SELECT имена_столбцов from имя_таблицы [WHERE ...условия];
```

Часть оператора с условиями является необязательной (мы рассмотрим ее позже). По сути, требуется знать имена столбцов и имя таблицы, из которой извлекаются данные.

Например, чтобы извлечь имена и фамилии всех сотрудников, выполните следующую команду.

```
SELECT f_name, l_name from employee_data;
```

Оператор приказывает MySQL вывести все данные из столбцов `f_name` и `l_name`.

При ближайшем рассмотрении можно заметить, что данные представлены в том порядке, в котором они были введены. Более того, последняя строка указывает число строк в таблице — 21.

Чтобы вывести всю таблицу, можно либо ввести имена всех столбцов, либо воспользоваться упрощенной формой оператора SELECT.

```
SELECT * from employee_data;
```

Символ `*` в этом выражении означает 'ВСЕ столбцы'. Поэтому этот оператор выводит все строки всех столбцов.

Рассмотрим ещё один пример.

```
SELECT f_name, l_name, age from employee_data;
```

Выборка данных с помощью условий

Теперь более подробно рассмотрим формат оператора **SELECT**. Его полный формат имеет вид:

```
SELECT имена_столбцов from имя_таблицы [WHERE ...условия];"
```

В операторе **SELECT** условия являются необязательными. Оператор **SELECT** без условий выводит все данные из указанных столбцов.

Теперь перейдём к рассмотрению операторов сравнения.

Операторы сравнения = и !=

```
SELECT f_name, l_name from employee_data where f_name = 'Иван';
```

Этот оператор выводит имена и фамилии всех сотрудников, которые имеют имя **Иван**. Отметим, что слово **Иван** в условии заключено в одиночные кавычки. Можно использовать также двойные кавычки. Кавычки являются обязательными, так как MySQL будет порождать ошибку при их отсутствии. Кроме того сравнения MySQL не различают регистр символов, что означает, что с равным успехом можно использовать "Иван", "иван" и даже "ИвАн".

```
SELECT f_name, l_name from employee_data where title="программист";
```

Выбирает имена и фамилии всех сотрудников, которые являются программистами.

```
SELECT f_name, l_name from employee_data where age = 32;
```

Это список имен и фамилий всех сотрудников с возрастом 32 года. Вспомните, что тип столбца **age** был задан как **int**, поэтому кавычки вокруг 32 не требуются. Это — незначительное различие между текстовым и целочисленным типами столбцов.

Оператор **!=** означает 'не равно' и является противоположным оператору равенства.

Операторы больше и меньше

Давайте получим имена и фамилии всех сотрудников, которые старше 32 лет.

```
SELECT f_name, l_name from employee_data where age > 32;
```

Попробуем найти сотрудников, которые получают зарплату больше 120000.

```
SELECT f_name, l_name from employee_data where salary > 120000;
```

Теперь перечислим всех сотрудников, которые имеют стаж работы в компании менее 3 лет.

```
SELECT f_name, l_name from employee_data where yos < 3;
```

Операторы <= и >=

Используемые в основном с целочисленными данными операторы меньше или равно (**<=**) и больше или равно (**>=**) обеспечивают дополнительные возможности.

```
select f_name, l_name, age, salary from employee_data where age >= 32;
```

Выборка содержит имена, возраст и зарплаты сотрудников, которым больше 32 лет.

```
select f_name, l_name from employee_data where yos <= 2;
```

Запрос выводит имена сотрудников, которые работают в компании не более 2 лет.

Поиск текстовых данных по шаблону

В данной части мы рассмотрим поиск текстовых данных по шаблону с помощью предложения `where` и оператора `LIKE`.

Оператор сравнения на равенство (`=`) помогает выбрать одинаковые строки. Таким образом, чтобы перечислить имена сотрудников, которых зовут *Иван*, можно воспользоваться следующим оператором `SELECT`.

```
select f_name, l_name from employee_data where f_name = "Иван";
```

Как быть, если надо вывести данные о сотрудниках, имя которых начинается с буквы *В*? Язык SQL позволяет выполнить поиск строковых данных по шаблону. Для этого в предложении `where` используется оператор `LIKE` следующим образом.

```
select f_name, l_name from employee_data where f_name LIKE "В%";
```

Можно видеть, что здесь в условии вместо знака равенства используется `LIKE` и знак процента в шаблоне.

Знак `\%` действует как символ-заместитель (аналогично использованию `*` в системах DOS и Linux). Он заменяет собой любую последовательность символов. Таким образом `В%` обозначает все строки, которые начинаются с буквы *В*. Аналогично `%В` выбирает строки, которые заканчиваются символом *В*, а `%В%` строки, которые содержат букву *В*.

Давайте выведем, например, всех сотрудников, которые имеют в названии должности строку *"про"*.

```
select f_name, l_name, title from employee_data where title like '%про%';
```

Перечислим всех сотрудников, имена которых заканчиваются буквой *'а'*. Это очень просто сделать.

```
select f_name, l_name from employee_data where f_name like '%а';
```

Предложение HAVING

Чтобы вывести среднюю зарплату сотрудников в различных подразделениях (должностях), используется предложение `GROUP BY`, например:

```
select title, AVG(salary) from employee_data GROUP BY title;
```

Предположим теперь, что требуется вывести только те подразделения, где средняя зарплата более 100000. Это можно сделать с помощью предложения `HAVING`.

```
select title, AVG(salary) from employee_data GROUP BY title HAVING AVG(salary) > 100000;
```

Удаление записей из таблицы

Для удаления записей из таблицы можно использовать оператор `DELETE`.

Оператор удаления `DELETE` требует задания имени таблицы и необязательных условий.

```
DELETE from имя_таблицы [WHERE условия];
```

Примечание: Если никакие условия не будут заданы, то удаляются все данные в таблице.

Предположим, один из специалистов по мультимедиа *'Василий Пупкин'* уволился из компании. Надо удалить его запись.

```
DELETE from employee_data WHERE l_name = 'Пупкин';
```

Задания

1. Напишите оператор SQL для создания новой базы данных с именем **addressbook**
2. Какой оператор используется для получения информации о таблице? Как используется этот оператор?
3. Как получить список всех баз данных, доступных в системе?
4. Напишите оператор для записи следующих данных в таблицу **employee_data**
Имя: Рудольф
Фамилия: Курочкин
Должность: Программист
Возраст: 34
Стаж работы в компании: 2
Зарплата: 95000
Надбавки: 17000
email: rudolf@yandex.ru
5. Приведите две формы оператора **SELECT**, которые будут выводить все данные из таблицы **employee_data**.
6. Как извлечь данные столбцов **f_name**, **email** из таблицы **employee_data**?
7. Напишите оператор для вывода данных из столбцов **salary**, **perks** и **yos** таблицы **employee_data**.
8. Как узнать число строк в таблице с помощью оператора **SELECT**?
9. Как извлечь данные столбцов **salary**, **l_name** из таблицы **employee_data**?
10. Напишите оператор **SELECT** для извлечения идентификационного номера сотрудников, которые старше 30 лет.
11. Напишите оператор **SELECT** для извлечения имен и фамилий всех Web-разработчиков.
12. Что выведет следующий оператор **SELECT**:

```
SELECT * from employee_data where salary <=100000;
```
13. Как вывести зарплаты и надбавки сотрудников, которые получают в качестве надбавок более 16000?
14. Перечислите имена всех сотрудников (фамилия, а затем имя), которые занимают должность бухгалтера.
15. Перечислить всех сотрудников, фамилии которых начинаются с буквы Р.
16. Вывести имена всех сотрудников в отделе продаж.
17. Что выведет следующий оператор

```
SELECT f_name, l_name, salary from employee_data where f_name like '%к%';
```
18. Перечислить фамилии и должности всех программистов
19. Вывести подразделения и средний возраст, где средний возраст больше 30.